STEWARD SYSTEMS

BUILDING THE OPERATING SYSTEMS OF INSTITUTIONS.

STEWARD SYSTEMS BEHAVIOURAL INTELLIGENCE AGENT

A Nigerian-Contextualised LLM Agent for Review Simulation and Personalised Recommendation

Solution Paper · Team Steward Systems · DSN x BCT LLM Agent Challenge / Data & AI Summit Hackathon 3.0

Abstract

This paper presents the Steward Systems Behavioural Intelligence Agent, a lightweight, reproducible, and explainable agentic system developed for the DSN x BCT LLM Agent Challenge / Data & AI Summit Hackathon 3.0. The system addresses both required tasks: user modelling with review simulation, and personalised recommendation. It is implemented as a FastAPI application with local sample data, deterministic fallback behaviour, transparent scoring, evaluation scripts, Docker configuration, optional semantic similarity support, and final API guardrails for judge-side reliability. The system is intentionally Nigerian-contextualised through a dedicated adapter that interprets affordability, value for money, delivery time, portion size, spice tolerance, local food and drink cues, Lagos-style delivery expectations, and mild Nigerian English tone. On the bundled reproducible sample data, Task A achieves RMSE 0.703 and MAE 0.546 for rating prediction. Task B achieves Hit Rate@5 0.429, Hit Rate@10 0.571, NDCG@5 0.155, and NDCG@10 0.207 under deterministic held-out evaluation. These figures are not presented as production-scale claims; rather, they demonstrate a sound architecture, transparent reasoning, reproducible evaluation, and a clear path to larger Yelp, Amazon Reviews, Goodreads, restaurant, or delivery-platform datasets.

Signal	Summary
Competition scope	Task A review/rating simulation; Task B personalised recommendation
Implementation posture	FastAPI, local data, deterministic fallback, Docker-ready, no required paid API key
Differentiator	Nigerian-contextual reasoning over price, portion, spice, delivery, and tone
Final reliability polish	Swagger placeholder guardrails, validation, integer star rating, safer explanations

1.0 Introduction

Modern review and recommendation systems should do more than map users to items by broad category. They should model behaviour: how strict a user is with ratings, what a user repeatedly praises, what a user complains about, the situations in which the user consumes an item, and the cultural expectations that shape those judgements. In a Nigerian setting, a useful recommendation may depend on whether the system understands price sensitivity, Lagos delivery delay, pepper tolerance, portion size, family or group consumption, and the practical meaning of value for money.

The Steward Systems Behavioural Intelligence Agent was built as a competition-grade foundation rather than a full SaaS product. It deliberately avoids external database dependencies, authentication, billing, multi-tenancy, and paid model requirements. The goal is to give judges a system they can run from a fresh clone, inspect through OpenAPI documentation, evaluate with scripts, and understand through transparent reasoning outputs.

The application covers the two required tasks. For Task A, it predicts a rating and generates a realistic review from user persona/history and target item details. For Task B, it produces personalised ranked recommendations from persona, optional history, current context, and candidate metadata. The central design choice is to combine deterministic behavioural modelling with agentic orchestration, rather than relying on a single opaque prompt.

2.0 Problem Formulation

2.1 Task A: User Modelling and Review Simulation

Given a user persona, optional past reviews, and a target item, the system predicts a star rating from 1 to 5 and generates a written review that is behaviourally faithful. Behavioural fidelity means the response should reflect rating tendency, strictness, preferences, complaints, tone, and contextual expectations. A generic review is insufficient; the output must explain why this particular user would likely respond to this particular item in this particular way.

The Task A response includes predicted_rating, predicted_star_rating, generated_review, behavioural_reasoning_summary, positive_signals, negative_signals, confidence, and user_profile_summary. The additional predicted_star_rating field provides a human-friendly integer interpretation while preserving the numeric score for evaluation.

2.2 Task B: Personalised Recommendation

Given a user persona, optional past reviews, optional current context, optional domain/category, and top_k, the system ranks candidate items. The output is designed to be useful and explainable rather than a flat list of names. Each recommendation includes rank, item identifiers, final score, score breakdown, reason, context fit, preference explanation, penalty explanation where applicable, and a cold-start note when the user has little or no history.

The problem is therefore not merely collaborative filtering. The agent must reason across behavioural evidence, item metadata, cultural context, and the immediate intent expressed in the current user situation.

3.0 Data and Preprocessing

The repository also includes optional Yelp Open Dataset ingestion support. The full Yelp dataset is not bundled because of size, but the ingestion script maps Yelp business, review, and user JSON files into the same internal user-item-review schema used by the agent. This allows the default container to remain lightweight while preserving compatibility with real-world review datasets.

The bundled data is synthetic/demo data designed for reproducibility and judge inspection. It is deliberately small enough to run locally and inside Docker while still covering diverse behaviours and item domains. The data is not presented as a production dataset; it is a controlled competition scaffold for demonstrating architecture, evaluation, and reasoning.

Data Type	Count
Users	18
Items	60
Reviews	160

The domains include food, drinks, books, movies/entertainment, and everyday products. Food and drinks are intentionally the strongest domain because Nigerian contextualisation is a competitive advantage for this challenge. Items include jollof rice, suya, shawarma, pepper soup, amala, egusi, fried rice, small chops, asun, zobo, Chapman, meat pie, puff-puff, bole, and grilled chicken.

The dataset includes distinct user behaviours: price-sensitive student, busy professional, family-oriented parent, spice lover, health-conscious user, strict reviewer, generous reviewer, delivery-sensitive user, ambience-focused diner, book/movie preference user, everyday product buyer, and cold-start users with no history. Metadata includes price, spice level, delivery time, portion size, location, tags, popularity score, and average rating.

For Task B evaluation, the system uses a deterministic held-out split. For users with enough reviews, earlier reviews become profile history while later reviews become test relevance labels. Items already present in profile history are removed from the candidate pool to avoid recommending already-consumed items from the training profile.

4.0 System Architecture

The system is implemented as a Python FastAPI application. API routes are intentionally thin; business logic lives in agents and services. This keeps the project readable, testable, and extendable while avoiding unnecessary SaaS infrastructure.

Layer	Module(s)	Role
API	app/main.py, app/api/	HTTP endpoints and OpenAPI docs
Schemas	app/models/schemas.py	Pydantic request/response contracts

Agents	app/agents/	Task orchestration
User modelling	app/services/user_profile_builder.py	Behavioural profile extraction
Nigerian context	app/services/nigerian_context_adapter.py	Cultural/contextual scoring and explanations
Semantics	app/services/semantic_similarity_service.py	Optional embeddings and deterministic fallback similarity
Ranking	app/services/recommendation_ranking_service.py	Hybrid recommendation scoring
Evaluation	evaluation/	Metrics, ablation, and human rubric

The application exposes GET /, GET /health, POST /api/task-a/simulate-review, POST /api/task-b/recommend, and GET /docs. The default semantic mode is a deterministic TF-IDF-style fallback. Optional sentence-transformer support is prepared for sentence-transformers/all-MiniLM-L6-v2, but it is disabled by default so the project does not depend on internet access, model downloads, or paid API keys at runtime.

4.1 API Guardrails and Input Validation

The API documentation was also polished with realistic Swagger/OpenAPI request examples for both tasks. This reduces judge-side friction by allowing evaluators to test the endpoints with meaningful example payloads rather than generic schema placeholders.

A final API guardrail layer was added to improve judge-side reliability and prevent meaningless default values from being interpreted as behavioural evidence. During manual testing through the FastAPI Swagger interface, default placeholder values such as "string" could be submitted accidentally. Without validation, such placeholders could distort the user profile, produce weak review text, and generate misleading recommendation explanations.

To address this, the system now detects and rejects placeholder-only item names and categories in Task A with a validation error. Placeholder values such as "string", empty strings, null, and similar non-informative inputs are not treated as user preferences, item features, or contextual signals. Where user profile evidence is weak, the system reports limited profile evidence instead of hallucinating strong behavioural cues.

The review simulation endpoint also includes consistency checks: generated reviews must refer to the actual item name where available, low predicted ratings must include concrete negative signals, and the response includes both numeric predicted_rating and integer predicted_star_rating for clearer judge interpretation. For recommendations, placeholder-only persona or context values trigger honest cold-start explanations rather than false claims of semantic or contextual fit.

This guardrail layer improves reproducibility, protects the demo from accidental Swagger-default submissions, and makes the system safer for external evaluation without adding unnecessary infrastructure or database dependencies.

5.0 Task A Method

5.1 Behavioural Profile Extraction

The profile builder extracts average rating, rating strictness, preferred categories, category affinity, positive preference signals, negative complaint signals, tone markers, price sensitivity, delivery sensitivity, spice preference, portion preference, and Nigerian context signals. Profile extraction uses both persona text and past reviews. High-rated reviews contribute preference terms and category affinity, while low-rated reviews contribute complaint and penalty signals.

5.2 Rating Prediction

Rating prediction combines user average rating tendency, strictness adjustment, preference overlap with the target item, category affinity, item quality/popularity metadata, price fit, spice fit, delivery fit, portion fit, Nigerian context fit, and negative penalties such as high price or slow delivery. The output remains bounded from 1 to 5. For clarity, the API also returns predicted_star_rating, an integer 1-5 interpretation of the predicted score, so that both machine-readable scoring and human-friendly star ratings are available.

5.3 Review Generation

The review generator is deterministic but varied. It uses controlled tones such as short practical tone, warm enthusiastic tone, strict reviewer tone, budget-conscious tone, delivery-sensitive tone, and family/social tone. Reviews aim for 45-90 words by default. They mention concrete item attributes such as price, delivery time, portion size, spice level, and location. Mild Nigerian English is used only where natural; exaggerated slang is avoided.

6.0 Task B Method

Task B includes explicit cold-start handling. If user evidence is missing or placeholder-only, the recommender ignores non-informative profile fields and falls back to broad popularity, item quality, and Nigerian-context fit rather than pretending that personal history exists.

The recommendation agent builds a user profile, loads local candidate items, filters by optional domain when provided, scores candidates, and returns ranked recommendations. It is intentionally hybrid: no single feature is allowed to dominate blindly. The selected scoring formula is transparent and was tuned through a small, explainable grid search.

final_score = 0.18 semantic_similarity + 0.38 preference_match + 0.20 context_match + 0.10 quality/popularity + 0.10 Nigerian_context_match - 0.04 penalty

Component	Meaning
Semantic similarity	Similarity between persona/history/context and item text
Preference match	Match to extracted user preferences, categories, spice, price, and portion needs
Context match	Fit to situation such as dinner, office lunch, family meal, or budget meal
Quality/popularity	Sample item metadata score

Nigerian context	Price, delivery, portion, pepper, location, and local food/drink fit
Penalty	Strong conflicts such as high price for a price-sensitive user or slow delivery for a delivery-sensitive user

Cold-start users are handled through persona text, current context, item metadata, and Nigerian-context rules. Cross-domain recommendations are supported because candidate items span food, drinks, books, movies, and everyday products. In the final guardrail pass, the recommendation endpoint was also improved so that weak or placeholder-only inputs are described honestly as cold-start cases instead of being presented as confident semantic matches.

7.0 Nigerian Contextualisation Layer

The Nigerian context adapter is a dedicated service rather than scattered hardcoded strings. It interprets affordability, value for money, price sensitivity, delivery time and Lagos traffic expectations, portion size and sharing needs, pepper/spice preference, Lagos/mainland/island or broader location cues, local foods and drinks, and Nigerian English tone.

The adapter improves Task A by adjusting ratings and enriching review generation. It improves Task B by contributing nigerian_context_match and human-readable explanations. The goal is not gimmickry; it is practical cultural relevance. A recommendation for a price-sensitive student after lectures, for example, should reason differently from a recommendation for a family-oriented parent ordering for a group.

8.0 Evaluation

8.1 Task A Metrics

Metric	Value
Examples	160
RMSE	0.703
MAE	0.546
Average generated review length	70.0 words

8.2 Task B Metrics

Metric	Value
Users evaluated	7
Held-out examples	21
Hit Rate@5	0.429
Hit Rate@10	0.571
NDCG@5	0.155
NDCG@10	0.207

8.3 Ablation Comparison

Variant	HR@5	HR@10	NDCG@5	NDCG@10
Popularity baseline	0.286	0.429	0.143	0.169

Content-only baseline	0.000	0.286	0.000	0.048
Hybrid without Nigerian context	0.286	0.571	0.121	0.194
Hybrid with Nigerian context	0.429	0.571	0.155	0.207
Hybrid with semantic embeddings if available	0.429	0.571	0.155	0.207

The sentence-transformer variant currently reports the same metrics because the default runtime uses tfidf_fallback. If embeddings are installed and enabled, the same service interface can use sentence-transformer similarity. The repository also includes a human evaluation rubric that scores Task A on behavioural fidelity, rating-review consistency, tone realism, item specificity, and Nigerian contextual naturalness. Task B is scored on contextual relevance, usefulness, explanation quality, cold-start quality, cross-domain usefulness, and Nigerian contextual fit.

9.0 Results and Discussion

The Task A RMSE and MAE show that the deterministic rating model can approximate sample ratings while keeping outputs explainable. The generated review length is within a practical range for product and restaurant reviews, and qualitative outputs include concrete details rather than generic praise. The final validation pass also improved the reliability of the public API surface. Placeholder-only Swagger inputs are now rejected or safely handled, preventing non-informative values from entering behavioural profiles or generated explanations. This does not change the core metrics, but it improves the credibility and robustness of the submitted agent during manual judging.

For Task B, the hybrid model outperforms the popularity baseline on Hit Rate@10 and NDCG@10. It also outperforms the hybrid-without-Nigerian-context variant on Hit Rate@5, NDCG@5, and NDCG@10. This suggests that Nigerian contextual signals help place relevant items earlier in the ranking, especially where price, spice, delivery, and portion cues matter.

The absolute NDCG values remain modest. This is expected for a small synthetic dataset with limited behavioural history and sparse held-out relevance labels. The purpose of the evaluation is to demonstrate a realistic, reproducible protocol and show directional value from the hybrid design, not to claim production recommendation accuracy.

10.0 Limitations

The sample data is synthetic and lightweight. It is useful for reproducibility but not a substitute for real transaction, review, or clickstream data. The deterministic review generator is controllable and safe but less expressive than a carefully evaluated LLM generator. The semantic embedding layer is optional and not enabled by default to avoid fragile downloads. The recommendation evaluation uses ratings as implicit relevance labels, which is practical for the hackathon but less rich than direct user feedback.

The system also does not include a production database, authentication, online learning, or vendor availability signals. These are deliberate scope choices. The challenge requires a containerised agent, a clean repository, and a solution paper; it does not require a full SaaS platform.

11.0 Future Work

- Larger Yelp, Amazon Reviews, Goodreads, restaurant, or delivery-platform datasets.
- Cached sentence-transformer embeddings and vector indexes for stronger semantic retrieval.
- Optional LLM review generation with deterministic fallback and strict evaluation.
- Human evaluation collection from Nigerian reviewers and domain judges.
- Freshness, availability, distance, and vendor reliability signals.
- Improved train/test protocols using timestamps, user sessions, and multi-turn context.
- Short demo video and deployment packaging for public presentation.

12.0 Conclusion

The Steward Systems Behavioural Intelligence Agent is a reproducible, explainable, Nigerian-contextualised agent application that solves both required competition tasks. It combines behavioural user profiling, deterministic review simulation, hybrid recommendation scoring, optional semantic similarity, Nigerian context adaptation, evaluation scripts, ablation evidence, human-evaluation materials, API guardrails, and judge-friendly documentation.

The project is intentionally lightweight, but its modular architecture is ready for larger datasets and stronger models after the hackathon foundation is accepted. Its central contribution is not merely that it generates text or ranks items; it models preference, context, and local behaviour in a way that is inspectable, reproducible, and professionally defensible.

Repository Reproducibility Summary

Check	Status
Tests	14 passed after final guardrail polish
Health/docs	/health and /docs return 200
Docker	Dockerfile and docker-compose.yml included; verify with docker compose up --build
No secrets	Uses .env.example and deterministic fallback by default
Paper and demo materials	Solution paper, demo script, sample payloads, checklist, and human rubric included